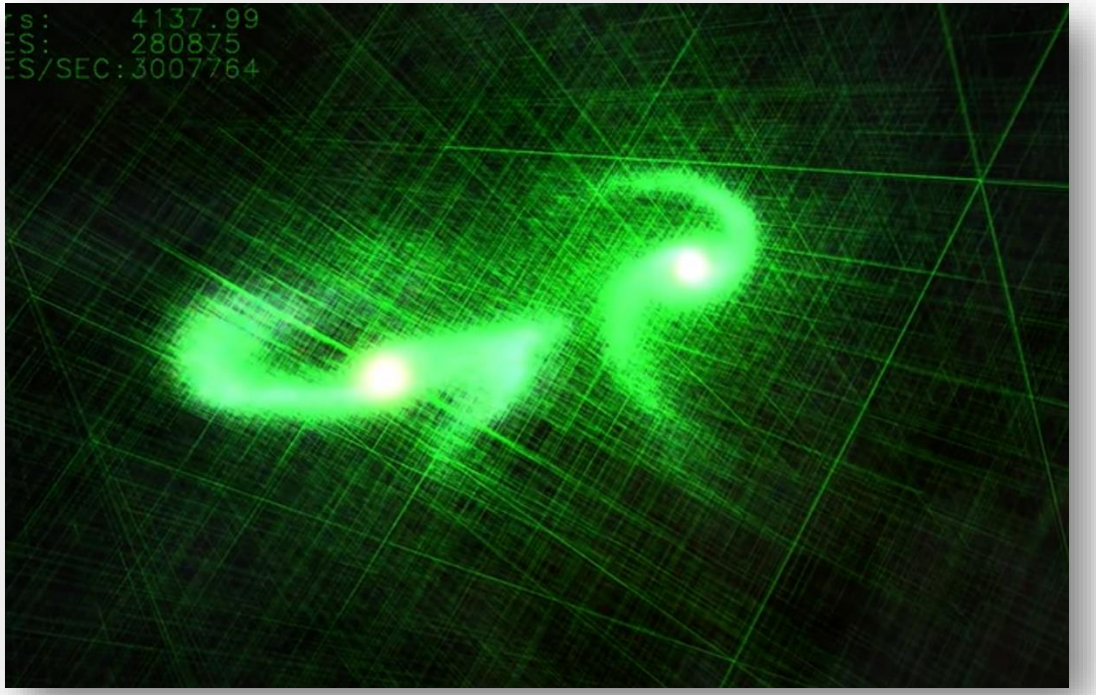
CSE113: Parallel Programming

- **Topics**:
  - Intro to concurrent data structures
  - Bank account example
  - Specification: Sequential consistency



Announcements

- Homework 2 is due on Thursday.
  - Plus 3 free late days
- Plenty of help: Piazza, office hours.
- You can share throughput numbers with each other. But don’t share code.

Announcements

- Homework 3 released on Thursday.

Announcements

- Starting module 3 today!

Previous quiz + Review

Previous quiz + Review

Which of the following are NOT ways that mutex implementations can encourage fair access?

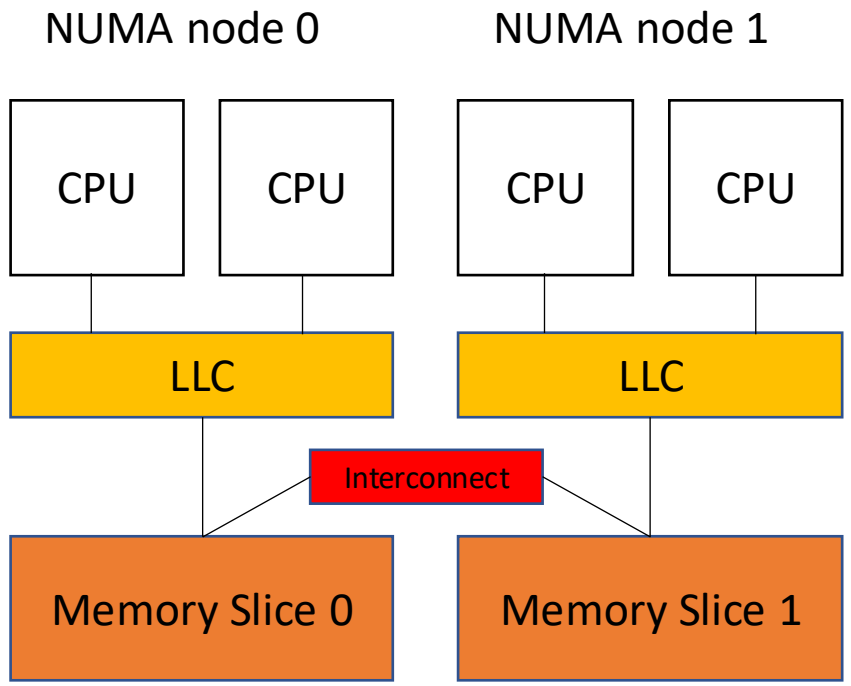
- ◯ Sleeping
- ◯ Yielding
- ◯ Using a ticket lock
- ◯ relaxed peeking

Optimizations: backoff

```
void lock(int thread_id) {
  bool e = false;
  bool acquired = false;
  while (!acquired) {
    while (flag.load(memory_order_relaxed) == true) {
      this_thread::yield();
    }
    e = false;
    acquired = atomic_compare_exchange_strong(&flag, &e, true);
  }
}
```

Optimization: Hierarchical locks

- Any sort of communication is very expensive:
  - Spinning triggers expensive coherence protocols.
  - cache flushes between NUMA nodes is expensive (transferring memory between critical sections)



```
void lock(int thread_id) {
  int e = -1;
  bool acquired = false;
  while (acquired == false) {
    acquired = atomic_compare_exchange_strong(&m_owner, &e, thread_id);

    if (thread_id/2 != e/2) {
      this_thread::sleep_for(10ms);
    }
    else {
      this_thread::sleep_for(1ms);
    }
    e = -1;
  }
}
```

Previous quiz + Review

A reader-writer mutex allows multiple readers in the critical section, mutliple writers in the critical section, but never a combination of readers and writers.

- ◯ True
- ◯ False

Previous quiz + Review

If you are an expert in how your code will compile to machine instructions, it is okay to have data conflicts in your code.

- ◯ True
- ◯ False

Previous quiz + Review

Why is the compare-and-swap operation required after the relaxed peeking sees that the mutex is available?

Optimizations: backoff

```
void lock(int thread_id) {
  bool e = false;
  bool acquired = false;
  while (!acquired) {
    while (flag.load(memory_order_relaxed) == true) {
      this_thread::yield();
    }
    e = false;
    acquired = atomic_compare_exchange_strong(&flag, &e, true);
  }
}
```

Final thoughts on Module 2

Data conflicts (Data race)

- Semantics of programs with data conflicts are undefined.
  - Compiler can do crazy things
  - interleavings cause bugs that are extremely rare
- Your code should use mutexes or atomics to avoid data conflicts!
- What happens when you don’t?

Horrible data conflicts in the real world

Therac 25: a radiation therapy machine

- Between 1987 and 1989 a software bug caused 6 cases where radiation was massively overdosed
- Patients were seriously injured and even died.
- Bug was root caused to be a data conflict.

- https://en.wikipedia.org/wiki/Therac-25
- https://www.youtube.com/watch?v=Ap0orGCiou8

Horrible data conflicts in the real world

2003 NE power blackout

- Second largest power outage in USA history: 55 million people were effected
- NYC was without power for 2 days, estimated 100 deaths
- Root cause was a data conflict

- https://en.wikipedia.org/wiki/Northeast_blackout_of_2003

Tools that Find Data Conflicts

Two approaches

- **Happens-before**: build a partial order of mutex lock/unlocks. Any memory accesses that can’t be ordered in this partial order is a conflict.

- **Lockset**: Every shared memory location is associated with a set of locks. Refine the lockset on every access. If a location is later accessed without the previous locks for the location, then report a conflict.

Dynamic Analysis

- Thread sanitizer:
  - a compiler pass built into Clang
  - About 10x overhead when you run the program
  - Identifies data conflicts and deadlocks

Static Analysis

- Meta Infer:
  - Statically checks for many issues (memory safety, assertions)
  - Can check for races in concurrent classes
  - Main support is for Java, although they claim support for C++

Research on data conflicts

- A recent tool:
  - Checks for C++ races
  - Scales to large programs
- Reports:
  - Chrome has 6 unresolved data-conflicts
  - Firefox has 52 unresolved data-conflicts
- Difficult to fix! 6.7 million lines of code in Chrome



Dynamic Race Detection for C++11

Christopher Lidbury
Imperial College London, UK
christopher.lidbury10@imperial.ac.uk

Alastair F. Donaldson
Imperial College London, UK
alastair.donaldson@imperial.ac.uk

Abstract

The intricate rules for memory ordering and synchronisation associated with the C/C++11 memory model mean that *data races* can be difficult to eliminate from concurrent programs. Dynamic data race analysis can pinpoint races in large and complex applications, but the state-of-the-art ThreadSanitizer (tsan) tool for C/C++ considers only sequentially consistent program executions, and does not correctly model synchronisation between C/C++11 atomic operations. We present a scalable dynamic data race analysis for C/C++11 that correctly captures C/C++11 synchronisation, and uses instrumentation to support exploration of a class of non sequentially consistent executions. We concisely define the memory model fragment captured by our instrumentation via a restricted axiomatic semantics, and show that the axiomatic semantics permits exactly those executions explored by our instrumentation. We have implemented our analysis in tsan, and evaluate its effectiveness on benchmark programs, enabling a comparison with the CDSChecker tool, and on two large and highly concurrent applications: the Firefox and Chromium web browsers. Our results show that our method can detect races that are beyond the scope of the original tsan tool, and that the overhead associated with applying our enhanced instrumentation to large applications is tolerable.

Categories and Subject Descriptors D.1.3 [*Programming Techniques*]: Concurrent Programming; D.2.5 [*Software Engineering*]: Debugging

concurrency, C++11, memory models

Another subtlety of this new memory model is the *reads-from* relation, which specifies the values that can be observed by an atomic load. This relation can lead to non-sequentially consistent (SC) behaviour; such *weak* behaviour can be counter-intuitive for programmers. The definition of *reads-from* is detailed and fragmented over several sections of the standards, and the weak behaviours it allows complicate data race analysis, because a race may be dependent upon a weak behaviour having occurred.

The aim of this work is to investigate the provision of automat
tool support for race analysis of C++11 programs, with the goa
helping C++11 programmers write race-free programs. The cu
state-of-the-art in dynamic race analysis for C++11 is Thread
tizer [43] (tsan). Although tsan can be applied to programs th
C++11 concurrency, the tool does not understand the spec
the C++11 memory model: it can both miss data races an
and report false alarms. The example programs of Figur
trate these issues: Figure 1a has a data race that tsan is in
detecting; Figure 1b has an assertion that can only fail
SC behaviour and hence cannot be explored by tsan;
free from data races due to C++11 fence semantics, b
racy by tsan. We discuss these examples in more det

In light of these limitations, the main research
consider are: (1) Can synchronisation properties c
gram be efficiently tracked during dynamic analys
a fragment of the C++11 memory model can
ciently during dynamic analysis? (3) Following
can we engineer a memory model-aware dy
tool that scales to large concurrent applicatio
fox and Chromium web browsers? These ap
be analysed using tsan, without the full exte
ory model; our question is whether by mo
aware of the memory model, we can still e

The programs we wish to analyse can
running concurrently, executing thousa

Summary

- Avoid data conflicts! They can cause serious bugs that trigger very very very rarely. (heisenbugs).
  - Better to use too many mutexes than not enough

- Use tools to help you!
  - Infer can helps with Java
  - Thread sanitizer helps with C++

On to new stuff!

Concurrent object motivation

- Programming basics cover a set of primitives:
  - types: ints, floats, bools
  - functions: call stacks, recursion

Concurrent object motivation

- Programming basics cover a set of primitives:
  - types: ints, floats, bools
  - functions: call stacks, recursion

simple example:
We can understand this!

```c
//Fibonacci Series using Recursion
#include<stdio.h>
int fib(int n)
{
   if (n <= 1)
      return n;
   return fib(n-1) + fib(n-2);
}

int main ()
{
  int n = 9;
  printf("%d", fib(n));
  getchar();
  return 0;
}
```

https://www.geeksforgeeks.org/c-program-for-n-th-fibonacci-number/

Concurrent object motivation

- How does it look moving into a more complicated setting?

Concurrent object motivation

- How does it look moving into a more complicated setting?
  - Hello world Android app:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Log.d("MainActivity", "Hello World");
}
```

https://developer.android.com/codelabs/android-training-hello-world#7

Concurrent object motivation

- How does it look moving into a more complicated setting?
  - Hello world Android app:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Log.d("MainActivity", "Hello World");
}
```

what the heck is a bundle?

https://developer.android.com/codelabs/android-training-hello-world#7

Concurrent object motivation

- How does it look moving into a more complicated setting?
  - Hello world Android app:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Log.d("MainActivity", "Hello World");
}
```

what is this?

https://developer.android.com/codelabs/android-training-hello-world#7

Concurrent object motivation

- How does it look moving into a more complicated setting?
  - Hello world Android app:

- These are objects!

https://developer.android.com/codelabs/android-training-hello-world#7

Concurrent object motivation

- Objects are user-specified abstractions:
  - A collection of data (state) and methods (behavior) representing something more complicated than primitive types can express.

Concurrent object motivation

- Objects are user-specified abstractions:
  - A collection of data (state) and methods (behavior) representing something more complicated than primitive types can express.

- Examples:
  - Writing a video game? objects for enemies and players
  - Writing an IOS app? objects for buttons

Concurrent object motivation

- Objects are user-specified abstractions:
  - A collection of data (state) and methods (behavior) representing something more complicated than primitive types can express.

- Examples:
  - Writing a video game? objects for enemies and players
  - Writing an IOS app? objects for buttons

- Objects allow programmer productivity:
  - Abstraction
  - Encapsulation
  - Modular and Composable

Concurrent object motivation

- Objects are user-specified abstractions:
  - A collection of data (state) and methods (behavior) representing something more complicated than primitive types can express.

- Examples:
  - Writing a video game? objects for enemies and players
  - Writing an IOS app? objects for buttons

- Objects allow programmer productivity:
  - Abstraction
  - Encapsulation
  - Modular and Composable

- We would like objects in the concurrent setting!

Concurrent object motivation

- Note:
  - The foundations in this lecture are general, and can be widely applied to many different types of objects
  - We will focus on “container” objects, lists, sets, queues, stacks.
  - These are:
    - Practical - used in many applications
    - Well-specified - their sequential behavior is agreed on
    - Interesting implementations - great for us to study!

Conceptual examples

- Shopping list: Going shopping with roommates



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)



Consider two people splitting the work.

Conceptual examples

- Shopping list: Going shopping with roommates



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)

What can go wrong?



Consider two people splitting the work.

Conceptual examples

- Shopping list: Going shopping with roommates



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)

What can go wrong?

We end up with duplicates



Consider two people splitting the work.

Conceptual examples

- Shopping list: Going shopping with roommates



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)

What can go wrong?

We end up with duplicates

We end up missing an item



Consider two people splitting the work.

Conceptual examples

- Shopping list: Going shopping with roommates



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)

What can go wrong?

We end up with duplicates

We end up missing an item

If my roommate decides to go surfing, then I could get stranded!



Consider two people splitting the work.

Conceptual examples

- Shopping list: Going shopping with roommates

What kind of object is the list?



eggs
carrots
tortillas

Best case:
2x as fast (so we can get back to CSE113 homework)

What can go wrong?

We end up with duplicates

We end up missing an item

If my roommate decides to go surfing, then I could get stranded!



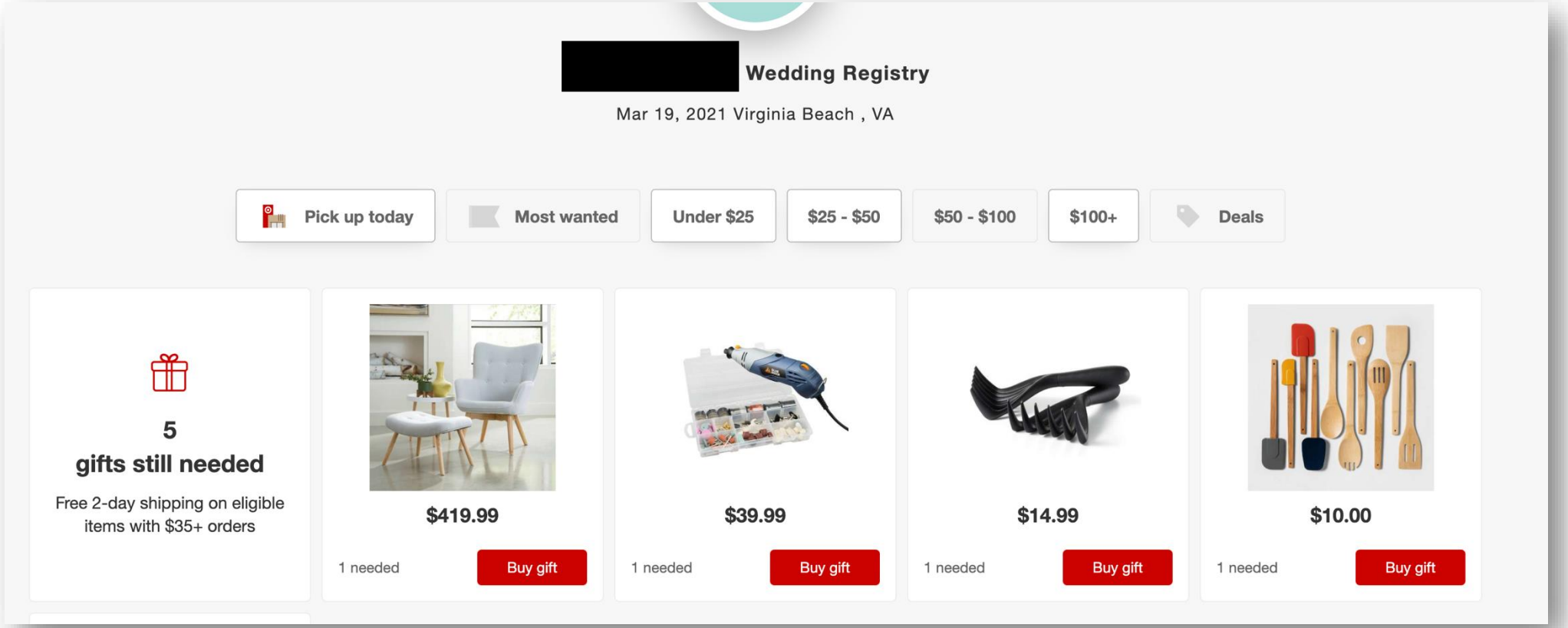
Consider two people splitting the work.

Conceptual examples

- Physically shopping with roommates is a nice conceptual example, but the example also occurs in automated systems

Conceptual examples

- Physically shopping with roommates is a nice conceptual example, but the example also occurs in automated systems



Shared memory concurrent objects

- Let’s ground this even more in a shared memory system.

- Shopping cart examples mostly occur in a distributed system setting where there are many different concerns

Shared memory concurrent objects

```
printf("hello world\n");
```

how do we envision printf to work?

```
printf("h");
printf("e");
printf("l");
printf("l");
printf("o");
```

terminal:
```
$ ./a.out
```

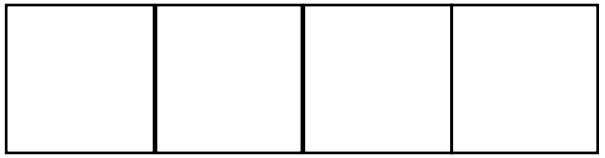
Shared memory concurrent objects

```
printf("hello world\n");
```

How does it actually work?

```
printf("h");
printf("e");
printf("l");
printf("l");
printf("o");
```

concurrent queue



```
terminal:
$ ./a.out
```

terminal display

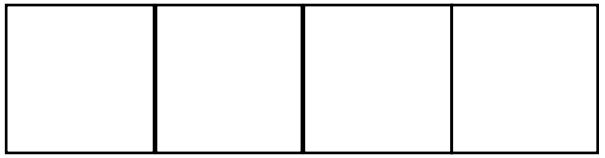
Shared memory concurrent objects

```
printf("hello world\n");
```

How does it actually work?

```
printf("h");
printf("e");
printf("l");
printf("l");
printf("o");
```

concurrent queue



A thread write, another reads.
Buffering helps performance.

```
terminal:
$ ./a.out
```

terminal display

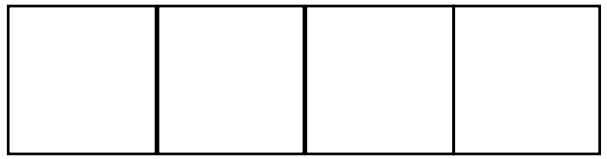
Shared memory concurrent objects

```
printf(“hello world\n”);
```

How does it actually work?

```
printf(“h”);
printf(“e”);
printf(“l”);
printf(“l”);
printf(“o”);
```

concurrent queue



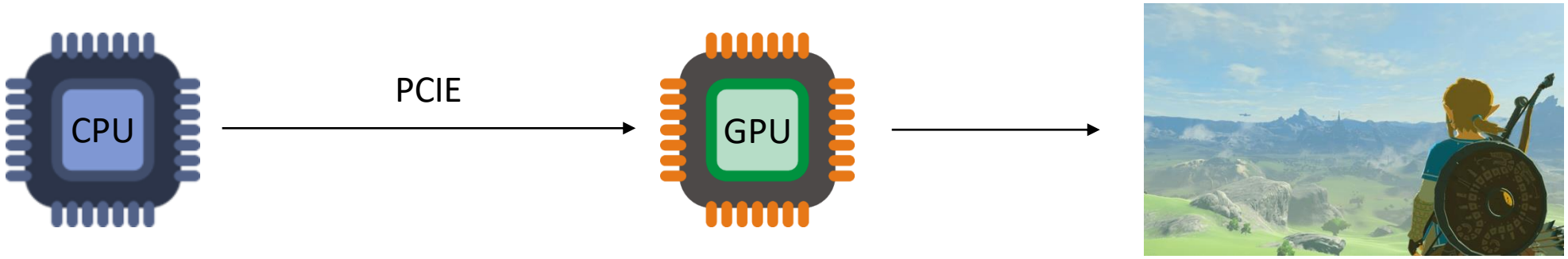
```
terminal:
$ ./a.out
```

terminal display

You can force a flush with: `fflush(stdout)`

Shared memory concurrent objects

- Graphics programming



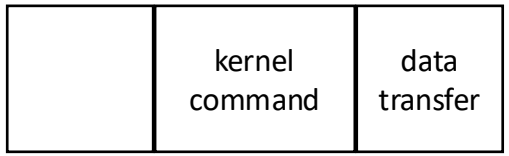
loop:
update data (data transfer)
graphics computation (kernel)

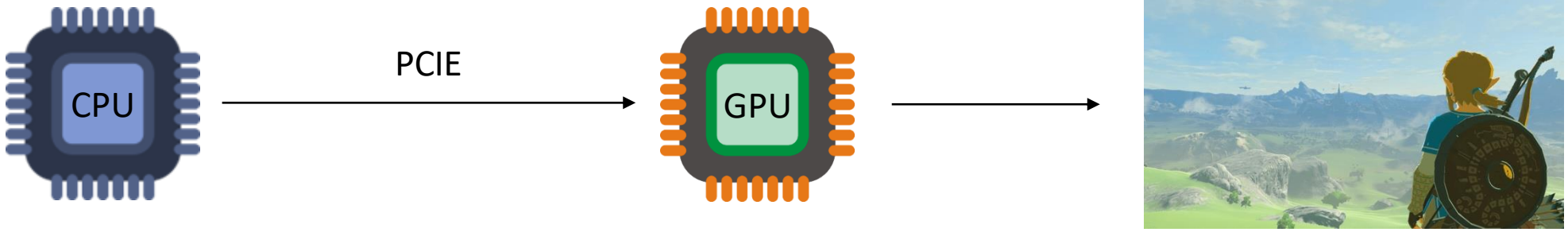
Nintendo: breath of the Wild

Shared memory concurrent objects

- Graphics programming

Vulkan/OpenCL CommandQueue





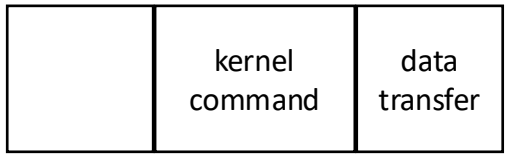
loop:
update data (data transfer)
graphics computation (kernel)

Nintendo: breath of the Wild

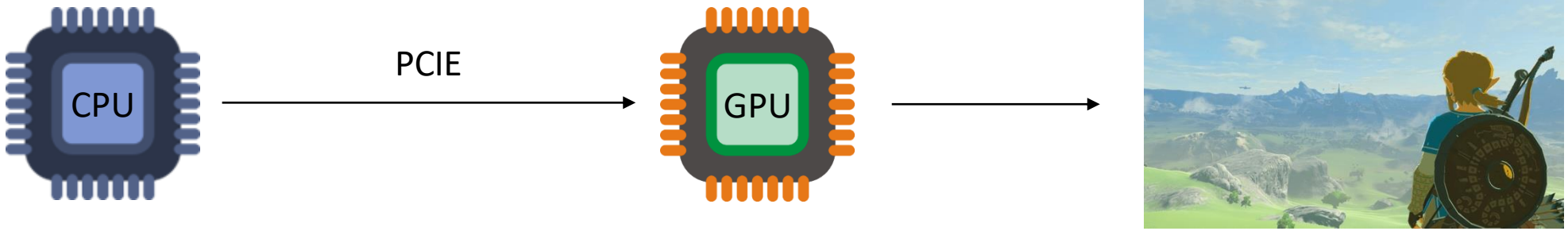
Shared memory concurrent objects

- Graphics programming

Vulkan/OpenCL CommandQueue



GPU driver concurrently reads from the queue



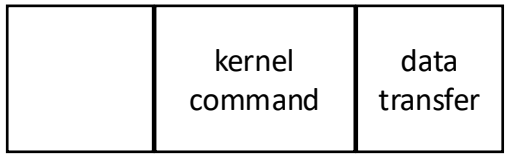
loop:
update data (data transfer)
graphics computation (kernel)

Nintendo: breath of the Wild

Shared memory concurrent objects

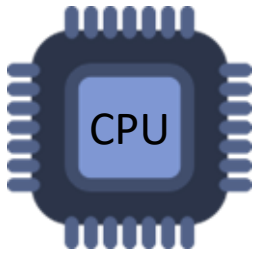
- Graphics programming

Vulkan/OpenCL CommandQueue



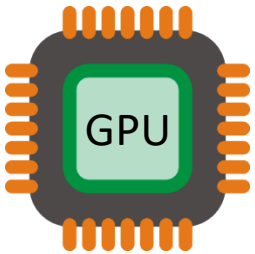
GPU driver concurrently reads from the queue

this concurrent queue enables an efficient graphics pipeline



PCIE

Transferring data for scene 2





loop:
update data (data transfer)
graphics computation (kernel)

Computation for scene 1

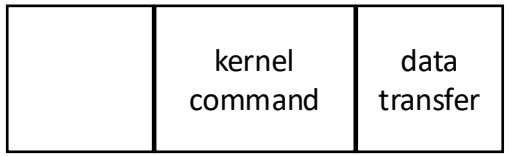
Scene 0

Nintendo: breath of the Wild

Shared memory concurrent objects

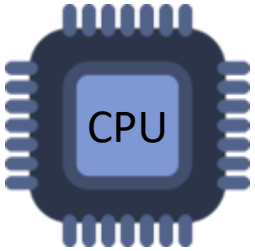
- Graphics programming

Vulkan/OpenCL CommandQueue



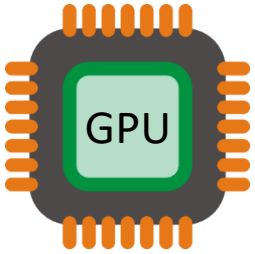
GPU driver concurrently reads from the queue

Single writer, single reader
Like in `printf`



PCIE

Transferring data for scene 2





Scene 0

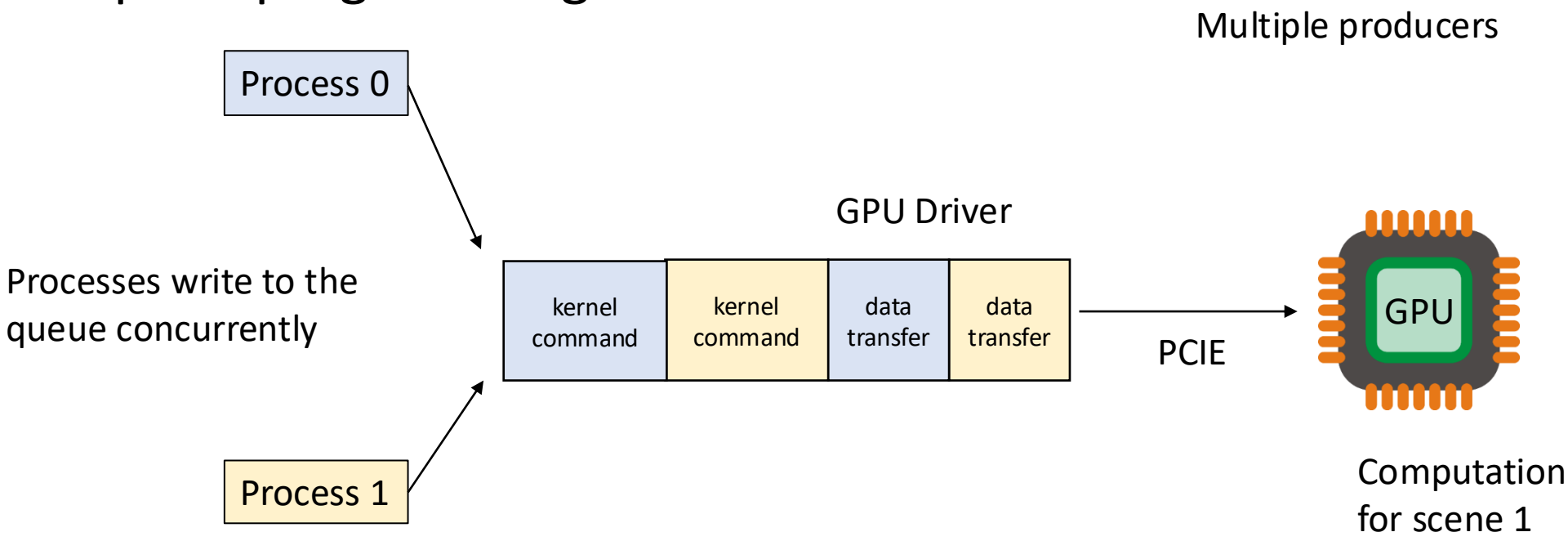
loop:
update data (data transfer)
graphics computation (kernel)

Computation for scene 1

Nintendo: breath of the Wild

Shared memory concurrent objects

- Graphics programming



Each process:
loop:
update data (data transfer)
graphics computation (kernel)

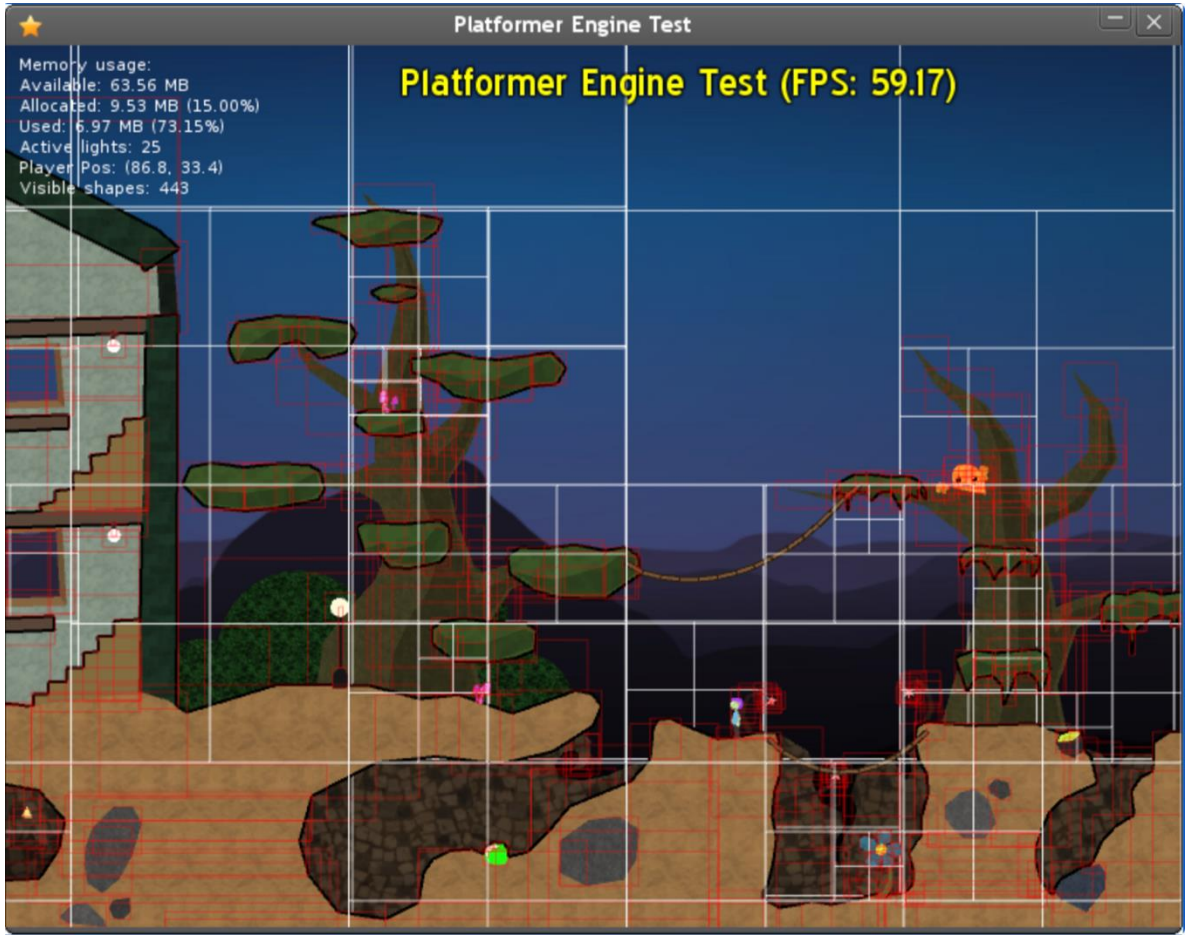
Intro to concurrent objects

- Prior examples have been infrastructural:
  - things happening behind the scenes, drivers, OS, etc.

- They also exist in standalone applications

Shared memory concurrent objects

- Quadtree
- *Octrees*
  the three-dimensional version of quadtrees

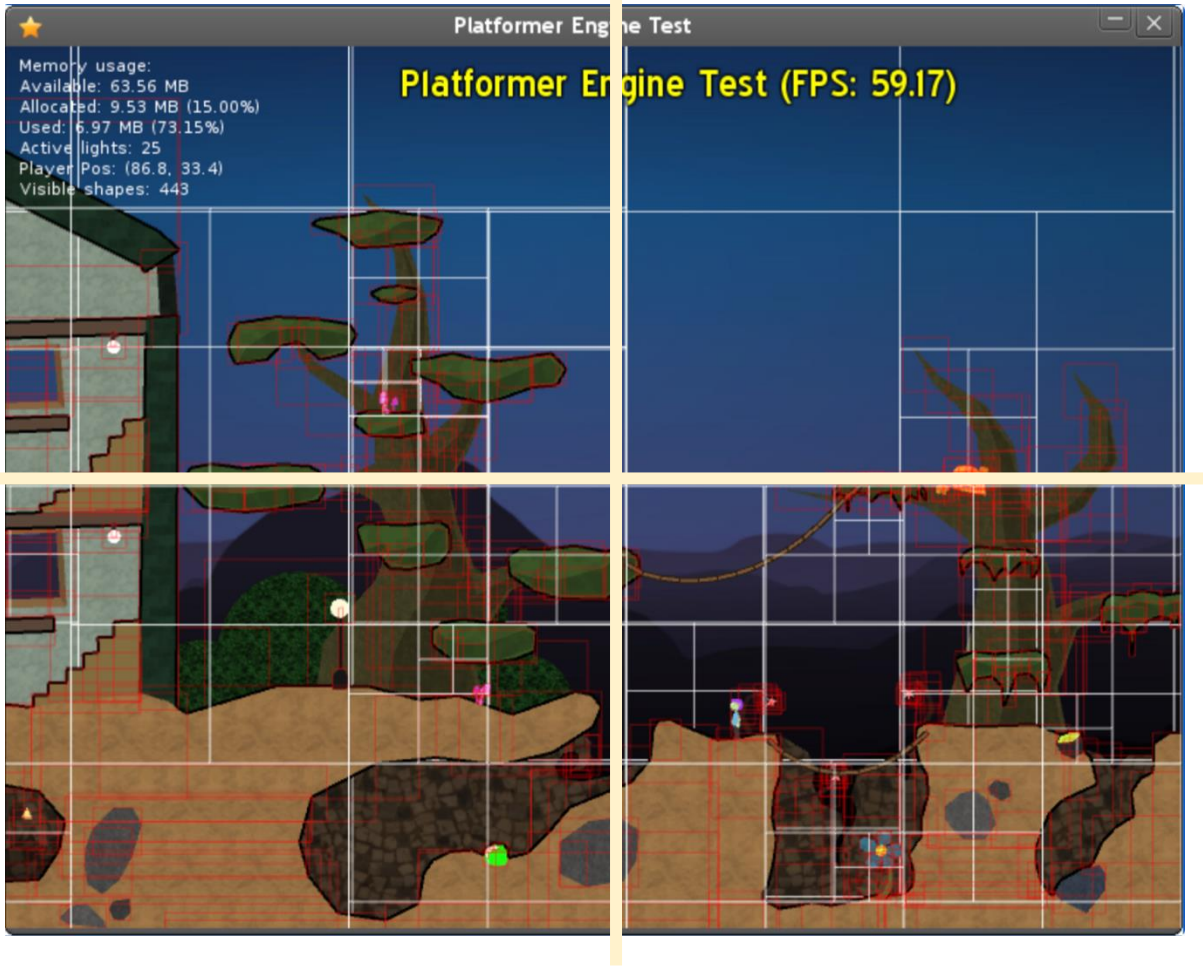


https://medium.datadriveninvestor.com/partitioning-2d-spaces-an-introduction-to-quadtrees-d95728856613

Shared memory concurrent objects

- Quadtree/Octree

recursively divide the scene giving more detail to “interesting” areas

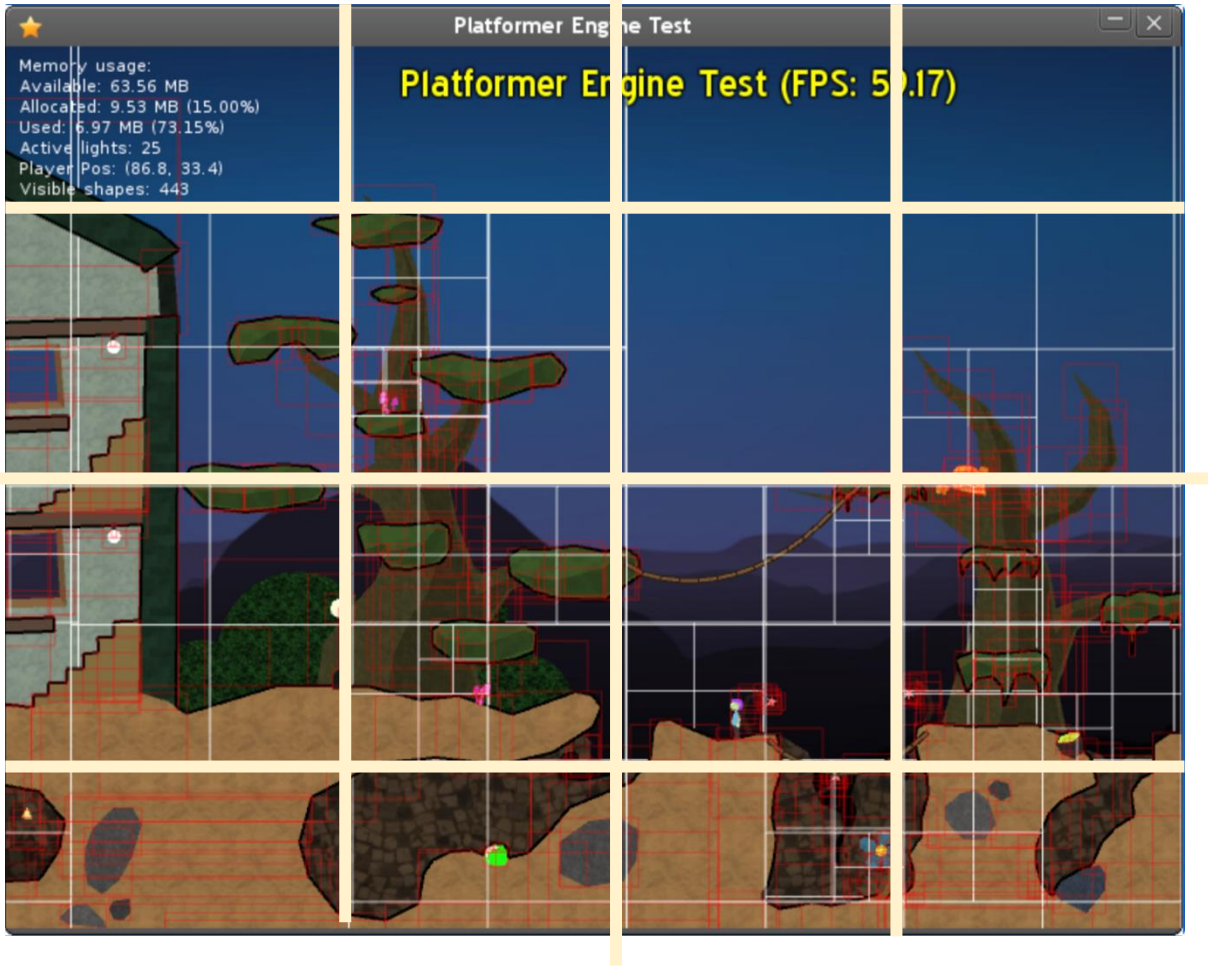


https://medium.datadriveninvestor.com/partitioning-2d-spaces-an-introduction-to-quadtrees-d95728856613

Shared memory concurrent objects

- Quadtree/Octree

recursively divide the scene giving more detail to “interesting” areas



https://medium.datadriveninvestor.com/partitioning-2d-spaces-an-introduction-to-quadtrees-d95728856613

Octree example

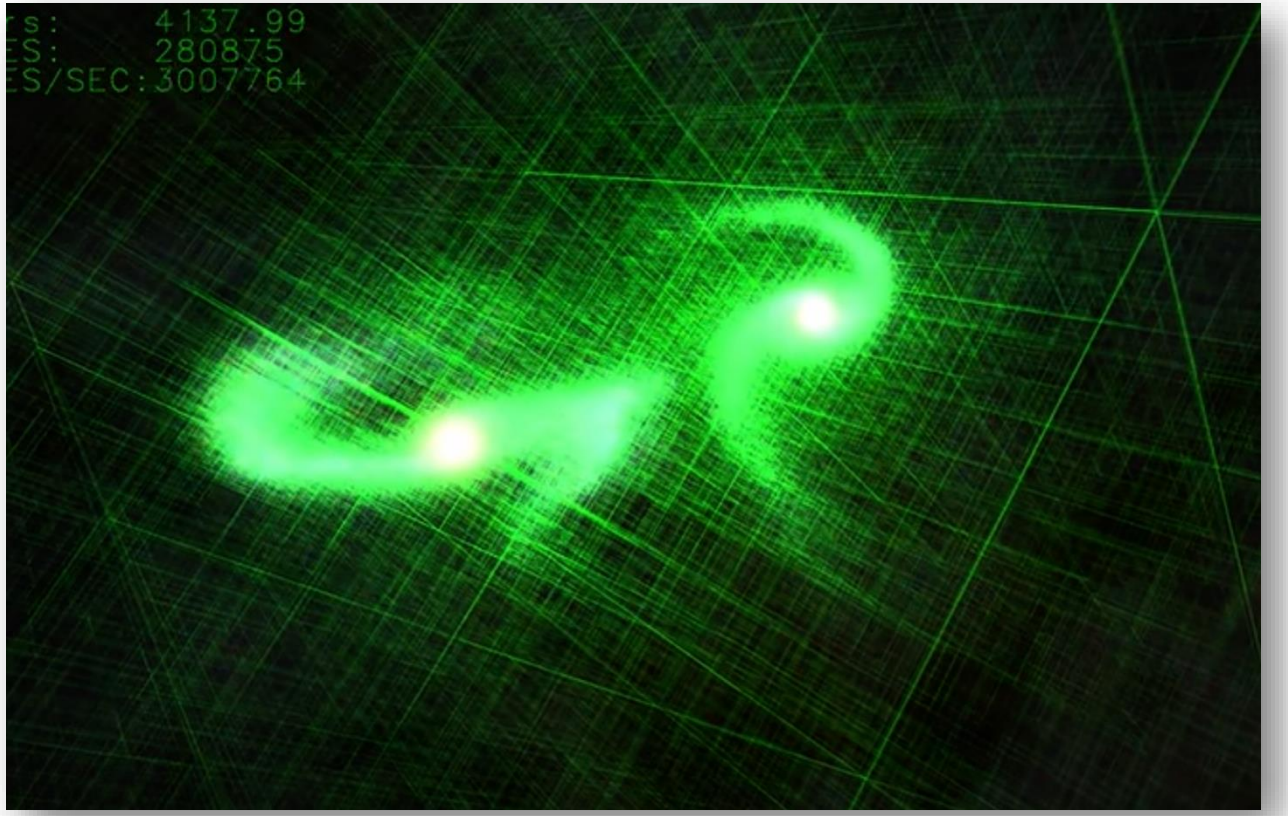
- Simulation of 2 galaxies colliding
- 280K stars



https://www.youtube.com/watch?v=aByz-mxOXJM

Octree example

- Simulation of 2 galaxies colliding
- 280K stars



https://www.youtube.com/watch?v=aByz-mxOXJM

Take away

- Concurrent data structures are everywhere!
  - Infrastructure
  - Applications

Schedule

- Intro to concurrent data structures
- **Bank account example**
- Specification: Sequential consistency

Bank account example

global variables:

```
int tylers_account = 0;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account -= 1;
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account += 1;
}
```

Bank account example

global variables:

```
int tylers_account = 0;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account -= 1;
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account += 1;
}
```

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

what happens if we run these concurrently?

Example

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

what happens if we run these concurrently?

Example

C++ will not magically make your objects concurrent!

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

The object is not "thread safe"

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

First solution:
The client (user of the object) can use locks.

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

The object is not "thread safe"

global variables:

```
bank_account tylers_account;
mutex m;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   m.lock();
   tylers_account.buy_coffee();
   m.unlock();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   m.lock();
   tylers_account.get_paid();
   m.unlock();
}
```

what if you have multiple objects?

First solution: The client (user of the object) can use locks.

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

The object is not "thread safe"

global variables:

```
bank_account tylers_account;
mutex m;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   m.lock();
   tylers_account.buy_coffee();
   m.unlock();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   m.lock();
   tylers_account.get_paid();
   m.unlock();
}
```

First solution:
The client (user of the object) can use locks.

client has to manage locks

We might decide to wrap my bank account in an object

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      balance -= 1;
    }

    void get_paid() {
      balance += 1;
    }

  private:
    int balance;
};
```

The object is not "thread safe"

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

we can encapsulate a mutex in the object.

The API stays the same!

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      m.lock();
      balance -= 1;
      m.unlock();
    }

    void get_paid() {
      m.lock();
      balance += 1;
      m.unlock();
    }

  private:
    int balance;
    mutex m;
};
```

Thread safe objects

- An object is thread-safe if you can call it concurrently
- Otherwise, you must provide your own locks!

Lock free programming

- An object is “lock free” if it is:
  - thread safe
  - does not use a lock in its underlying implementation.

- We can make a lock free bank account

```
atomic_fetch_add(atomic_int * addr, int value) {
   int tmp = *addr; // read
   tmp += value;    // modify
   *addr = tmp;     // write
}
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      m.lock();
      balance -= 1;
      m.unlock();
    }

    void get_paid() {
      m.lock();
      balance += 1;
      m.unlock();
    }

  private:
    int balance;
    mutex m;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      m.lock();
      balance -= 1;
      m.unlock();
    }

    void get_paid() {
      m.lock();
      balance += 1;
      m.unlock();
    }

  private:
    atomic_int balance;
    mutex m;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {

      balance -= 1;

    }

    void get_paid() {

      balance += 1;

    }

  private:
    atomic_int balance;
};
```

Bank account example

global variables:

```
bank_account tylers_account;
```

Tyler's coffee addiction:

```
for (int i = 0; i < HOURS; i++) {
   tylers_account.buy_coffee();
}
```

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account.get_paid();
}
```

```
class bank_account {
  public:
    bank_account() {
      balance = 0;
    }

    void buy_coffee() {
      atomic_fetch_add(&balance, -1);
    }

    void get_paid() {
      atomic_fetch_add(&balance, 1);
    }

  private:
    atomic_int balance;
};
```

How does it perform

How does it perform

- Better!
  - Mutexes reduce parallelism
  - Mutexes require many RMW operations

- Straight forward to do with the bank account, we will apply this to more objects

3 properties for concurrent objects

- **Correctness**:
  - How should concurrent objects behave (Specification)
- **Performance**:
  - How to make things fast fast fast!
- **Fairness**:
  - Under what conditions can concurrent objects deadlock

Let’s think about a Queue

What is a queue?

We consider 2 API functions:

- enq(value v) - enqueues the value v
- deq() - returns the value at the front of the queue

Sequential specification:

```
Queue<int> q;
q.enq(6);
int t = q.deq();
```

```
Queue<int> q;
q.enq(6);
q.enq(7);
int t = q.deq();
```

```
Queue<int> q;
q.enq(6);
q.enq(7);
int t = q.deq();
int t1 = q.deq();
```

Lets think about a Queue

What is a queue?

We consider 2 API functions:

- enq(value v) - enqueues the value v
- deq() - returns the value at the front of the queue

Sequential specification:

```
Queue<int> q;
int t = q.deq();
```

Let's say: *Error value of 0*

Lets think about a Queue

This is called a sequential specification:

The sequential specification is nice! We want to base our concurrent specification on the sequential specification

We will have to deal with the non-determinism of concurrency

Thinking about a concurrent queue

```
Queue<int> q;
q.enq(6);
q.enq(7);
int t = q.deq();
```

Thinking about a concurrent queue

Global variable:

```
CQueue<int> q;
```

Let's call our concurrent queue "CQueue"

Thread 0:

```
q.enq(6);
q.enq(7);
int t = q.deq();
```

Thinking about a concurrent queue

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

what can be stored in t after this concurrent program?

Thinking about a concurrent queue

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

what can be stored in t after this concurrent program?

Can t be 256?

Thinking about a concurrent queue

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

what can be stored in t after this concurrent program?

Can t be 256? it should be one of {None, 6}

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:

```
int t = q.deq();
```

```
q.enq(6);
```

```
q.enq(7);
```



t is 6

Global variable:

```
CQueue<int> q;
```

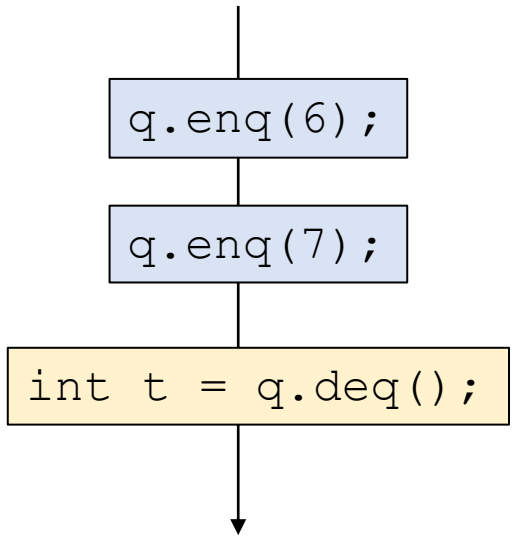
Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



t is 6

Global variable:

```
CQueue<int> q;
```

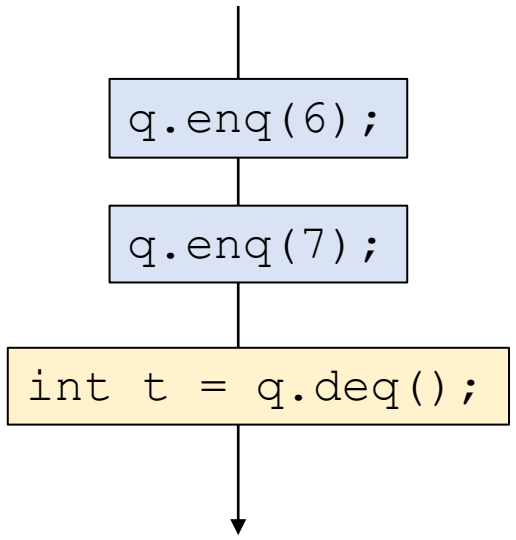
Thread 0:

```
q.enq(6);
q.enq(7);
```

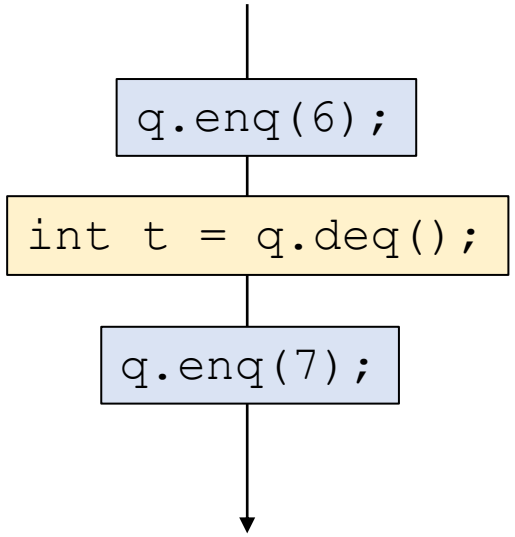
Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



t is 6



t is 6

Global variable:

```
CQueue<int> q;
```

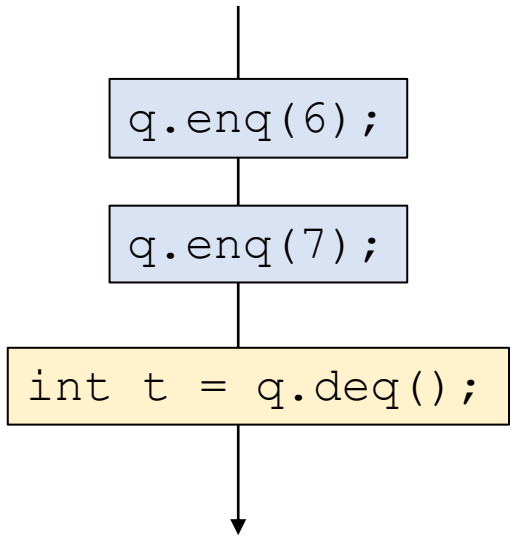
Thread 0:

```
q.enq(6);
q.enq(7);
```

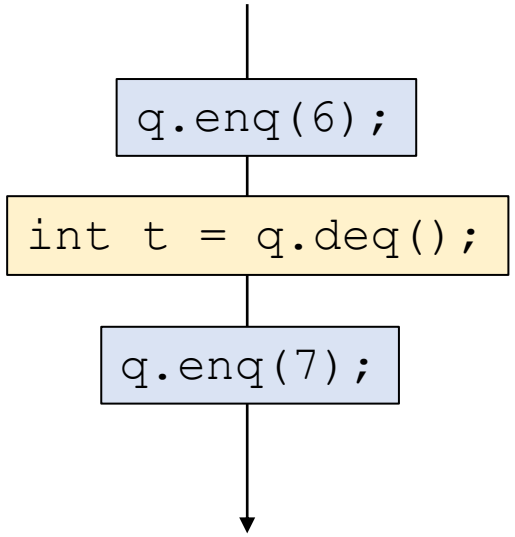
Thread 1:

```
int t = q.deq();
```

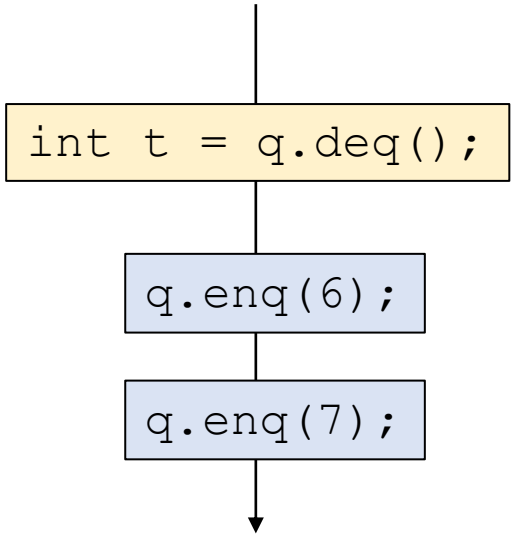
Construct a sequential timeline of API calls
Any sequence is valid:



t is 6



t is 6



t is None

Global variable:

```
CQueue<int> q;
```

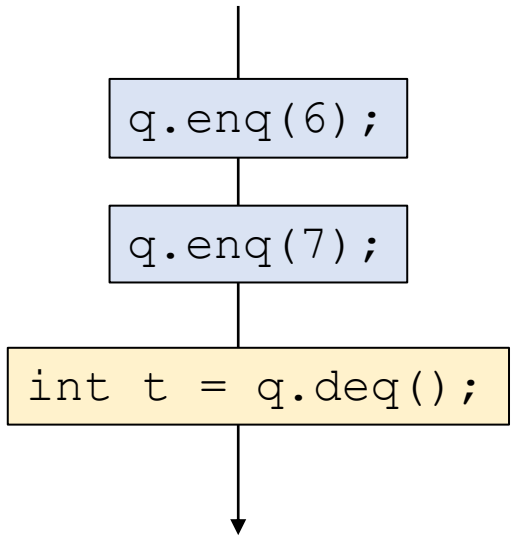
Thread 0:

```
q.enq(6);
q.enq(7);
```

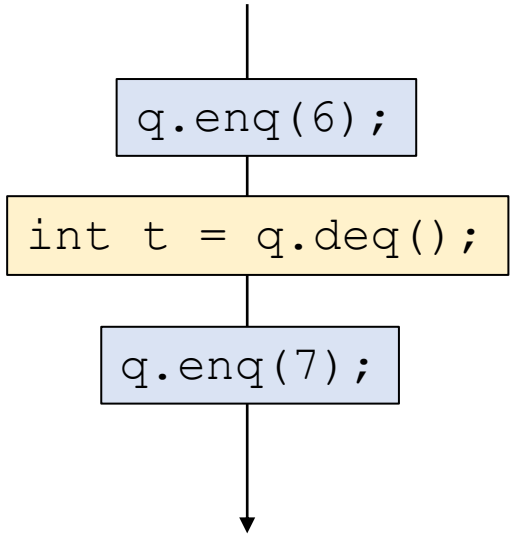
Thread 1:

```
int t = q.deq();
```

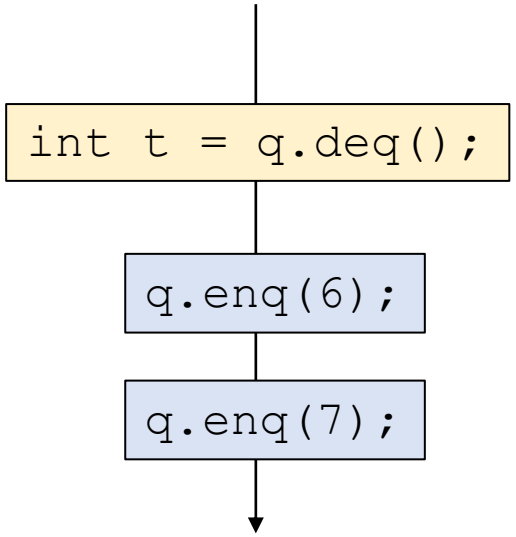
Construct a sequential timeline of API calls
Any sequence is valid:



t is 6



t is 6



t is None

Can t ever be 7?

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:

Can t ever be 7?

Global variable:

```
CQueue<int> q;
```

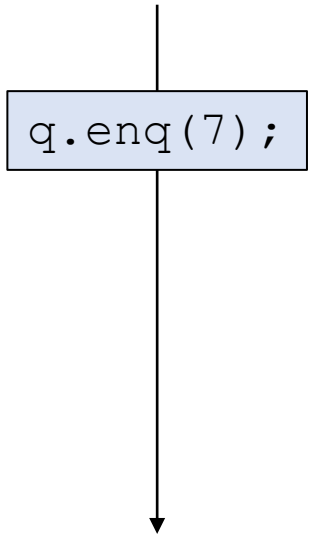
Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



Can t ever be 7?

Global variable:

```
CQueue<int> q;
```

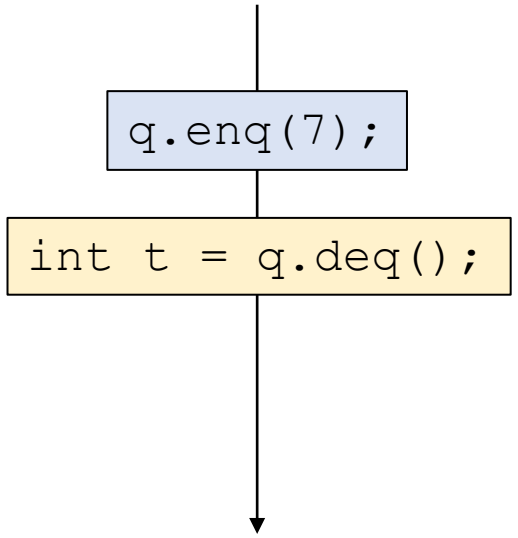
Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



Can t ever be 7?

Global variable:

```
CQueue<int> q;
```

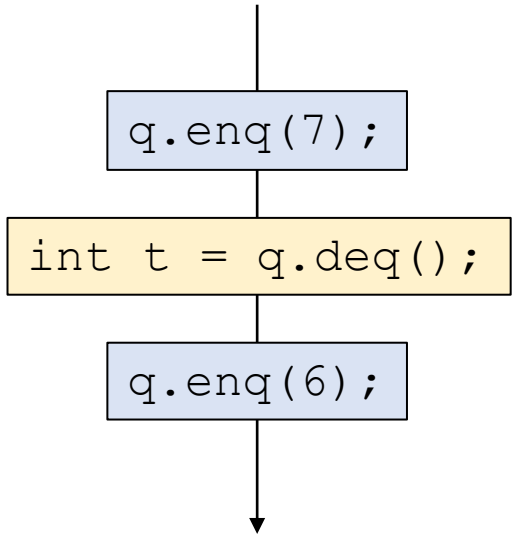
Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



Can t ever be 7?

Global variable:

```
CQueue<int> q;
```

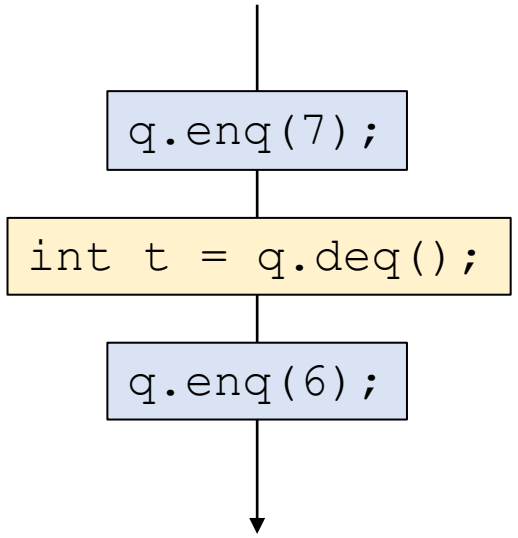
Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t = q.deq();
```

Construct a sequential timeline of API calls
Any sequence is valid:



The events of Thread 0 don't appear in the same order of the program!

This should not be allowed!

Can t ever be 7?

Sequential Consistency

- Valid executions correspond to a sequentialization of object method calls
- The sequentialization must respect per-thread ”program order”, the order in which the object method calls occur in the thread
- Events across threads can interleave in any way possible

Sequential Consistency

- Valid executions correspond a sequentialization of object method calls

- The sequentialization must respect per-thread ”program order”, the order in which the object method calls occur in the thread

- Events across threads can interleave in any way possible

How many possible interleavings?
Combinatorics question:

if Thread 0 has N events
if Thread 1 has M events

$$\frac{(N+M)!}{N!\,M!}$$

Sequential Consistency

How many possible interleavings?
Combinatorics question:

if Thread 0 has N events
if Thread 1 has M events

$$\frac{(N+M)!}{N!\,M!}$$

Reminder that N and M are events, not instructions

Sequential Consistency

How many possible interleavings?
Combinatorics question:

if Thread 0 has N events
if Thread 1 has M events

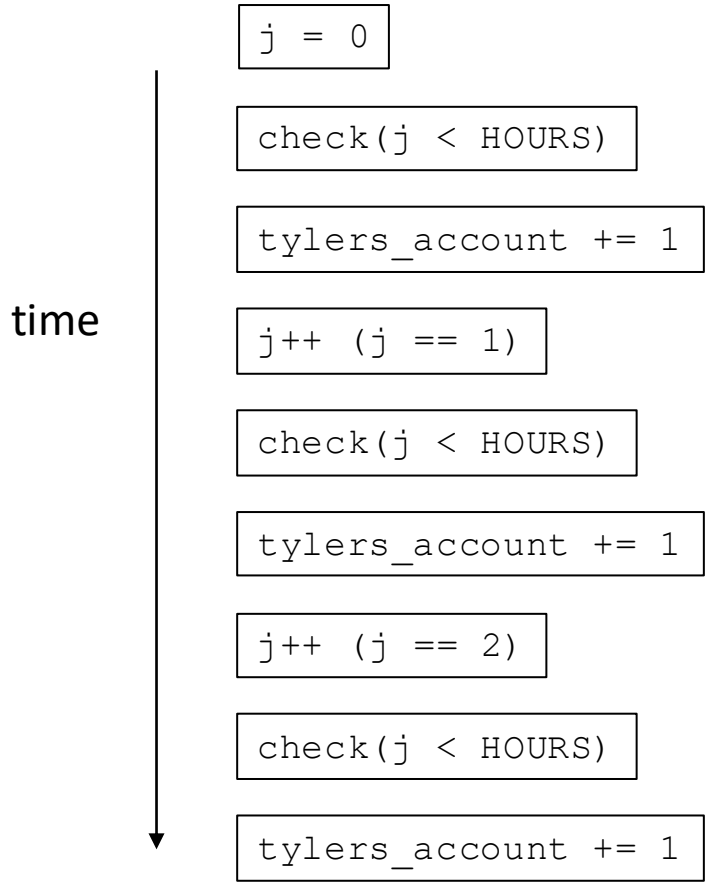
$$\frac{(N+M)!}{N!\,M!}$$

Reminder that N and M are events, not instructions

If N and M execute 150 events each, there are more possible executions than particles in the observable universe!

Tyler's employer

```
for (int j = 0; j < HOURS; j++) {
   tylers_account += 1;
}
```



Don’t think about all possible interleavings!

- Higher-level reasoning:
  - I get paid 100 times and buy 100 coffees, I should break even
  - If you enqueue 100 elements to a queue, you should be able to dequeue 100 elements

- Reason about a specific outcome
  - Find an interleaving that allows the outcome
  - Find a counter example

Reasoning about concurrent objects

To show that an outcome is possible, simply construct the sequential sequence

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```


Can t0 == 0 and t1 == 6?

Global variable:

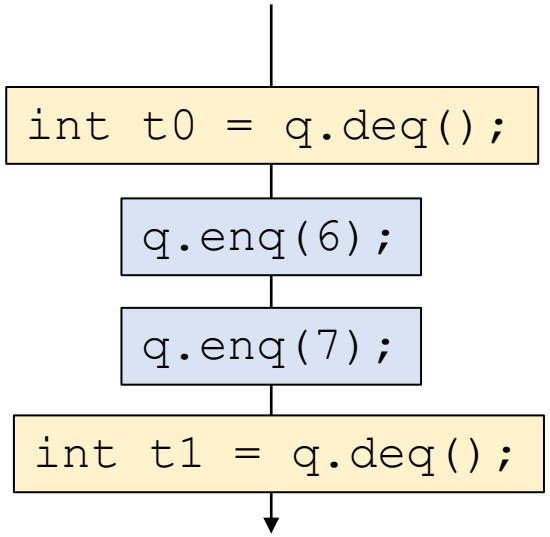
```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```



Can t0 == 0 and t1 == 6?

Global variable:

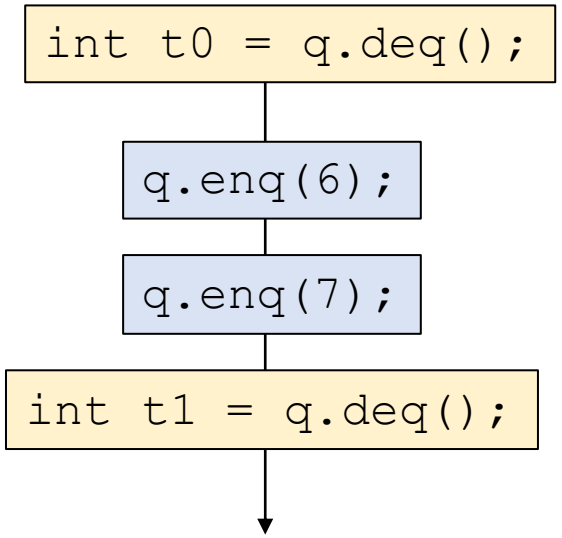
```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```



Can t0 == 0 and t1 == 6?

Valid execution!

Are there others?

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Lets do another!

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```

Can t0 == 6 and t1 == 7?

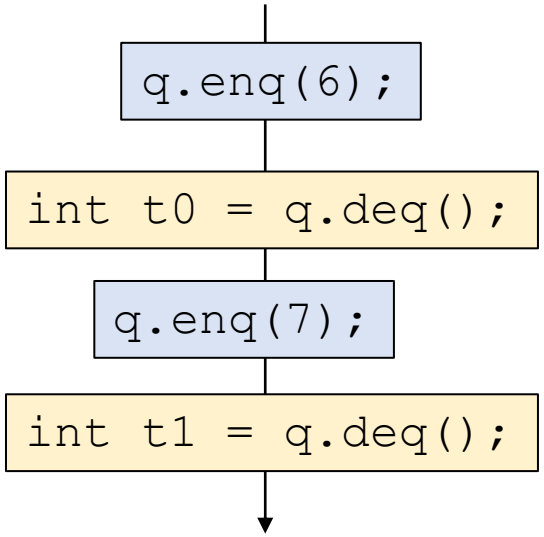
Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Lets do another!



Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```

Can t0 == 6 and t1 == 7?

Global variable:

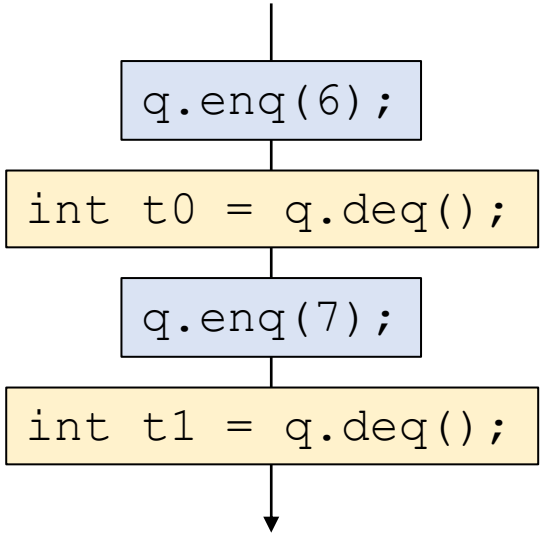
```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```



Can t0 == 6 and t1 == 7?

Found one! Are there others?

Reasoning about concurrent objects

To show that an outcome is possible, simply construct the sequential sequence

To show that an outcome is ***impossible*** show that there is no possible sequential sequence

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```


Can t0 == 0 and t1 == 7?

Global variable:

```
CQueue<int> q;
```

Thread 0:

```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```

```
q.enq(6);
```

```
q.enq(7);
```



```
int t0 = q.deq();
```

```
int t1 = q.deq();
```

Can t0 == 0 and t1 == 7?

Global variable:

```
CQueue<int> q;
```

Thread 0:

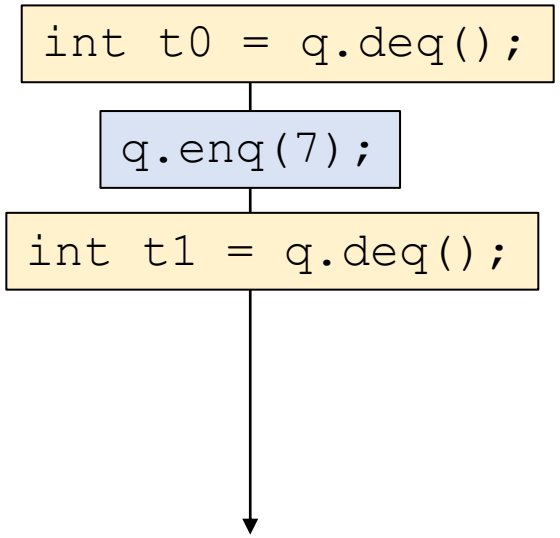
```
q.enq(6);
q.enq(7);
```

Thread 1:

```
int t0 = q.deq();
int t1 = q.deq();
```

```
q.enq(6);
```

No place for this event to go!



Can t0 == 0 and t1 == 7?

One more example

Global variable:

```
CStack<int> s;
```

Thread 0:

```
s.enq(7);
int t0 = q.dec();
```

Thread 1:

```
int t1 = q.dec();
```



Is it possible for both t0 and t1 to be 0 at the end?

Global variable:

```
CQueue<int> s;
```

Thread 0:

```
s.enq(7);
int t0 = q.dec();
```

Thread 1:

```
int t1 = q.dec();
```

```
q.enq(7);
```

```
int t0 = q.deq();
```

```
int t1 = q.dec();
```



Is it possible for both t0 and t1 to be 0 at the end?

Global variable:

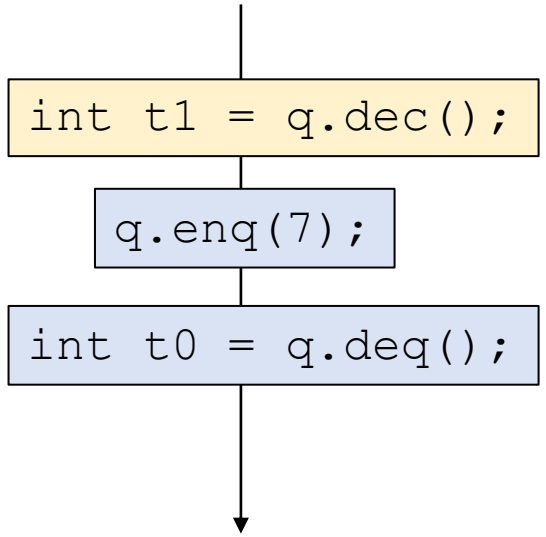
```
CQueue<int> s;
```

Thread 0:

```
s.enq(7);
int t0 = q.dec();
```

Thread 1:

```
int t1 = q.dec();
```



Is it possible for both t0 and t1 to be 0 at the end?

Global variable:

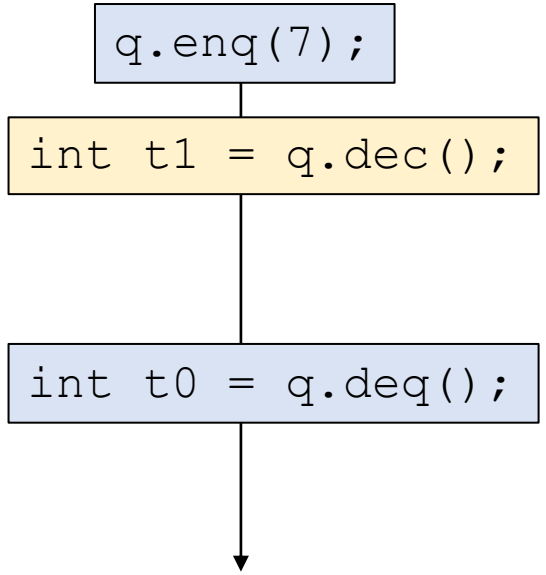
```
CQueue<int> s;
```

Thread 0:

```
s.enq(7);
int t0 = q.dec();
```

Thread 1:

```
int t1 = q.dec();
```



Is it possible for both t0 and t1 to be 0 at the end?

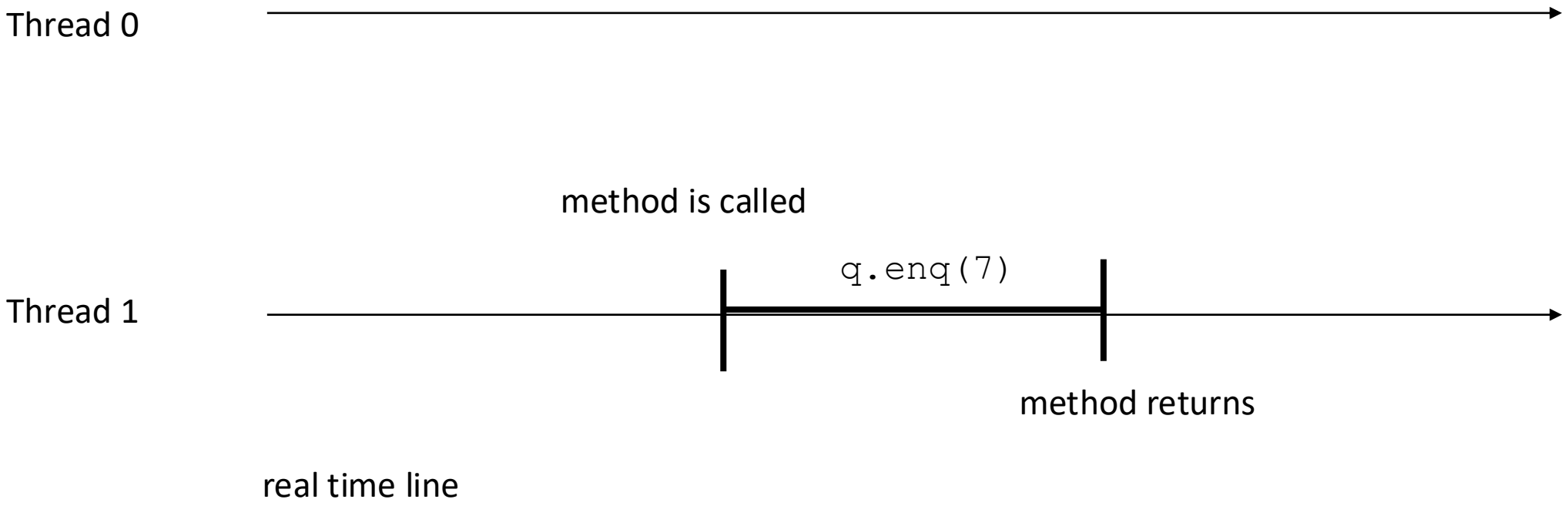
Do we have our specification?

- Is sequential consistency a good enough specification for concurrent objects?
- It’s a good first step, but relative timing interacts strangely with absolute time.
- We will need something stronger.

Sequential consistency and real time

- Add in real time:

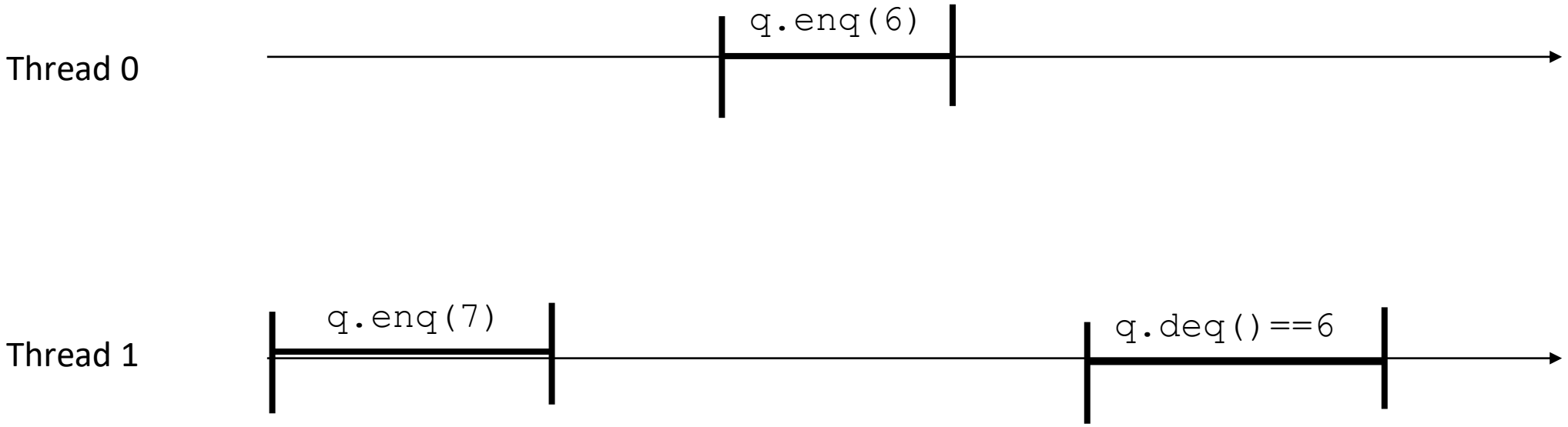
each method has a start, and end time stamp



Sequential consistency and real time

- Add in real time:

This timeline seems strange...

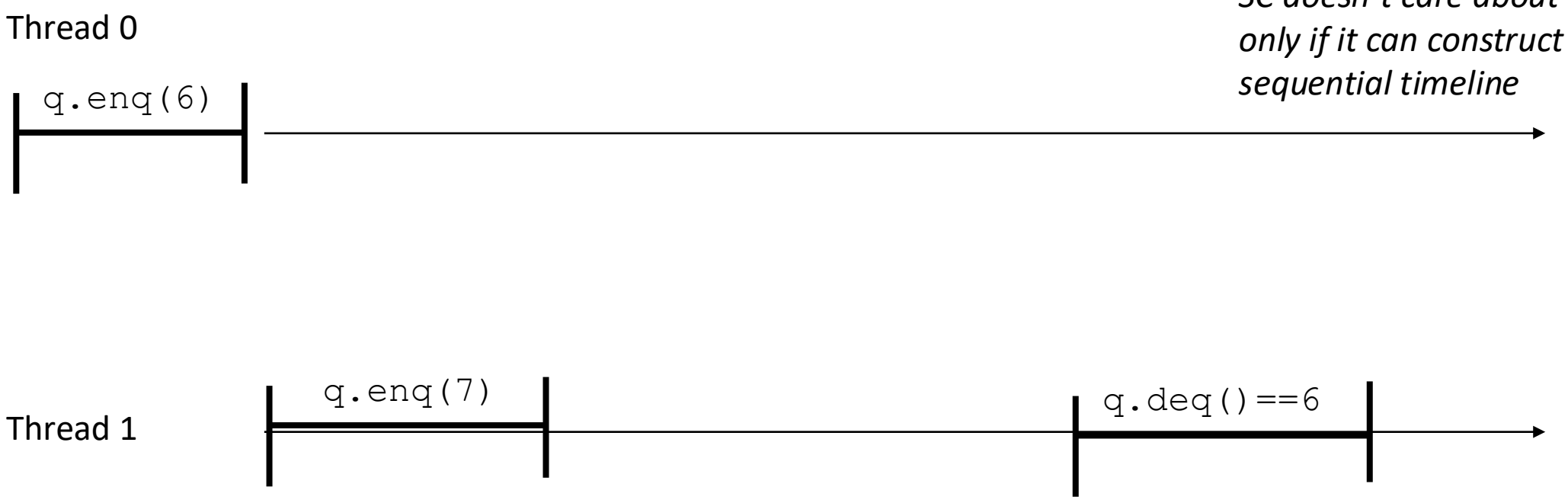


Sequential consistency and real time

- Add in real time:

This execution is allowed in sequential consistency!

SC doesn't care about real time, only if it can construct its virtual sequential timeline

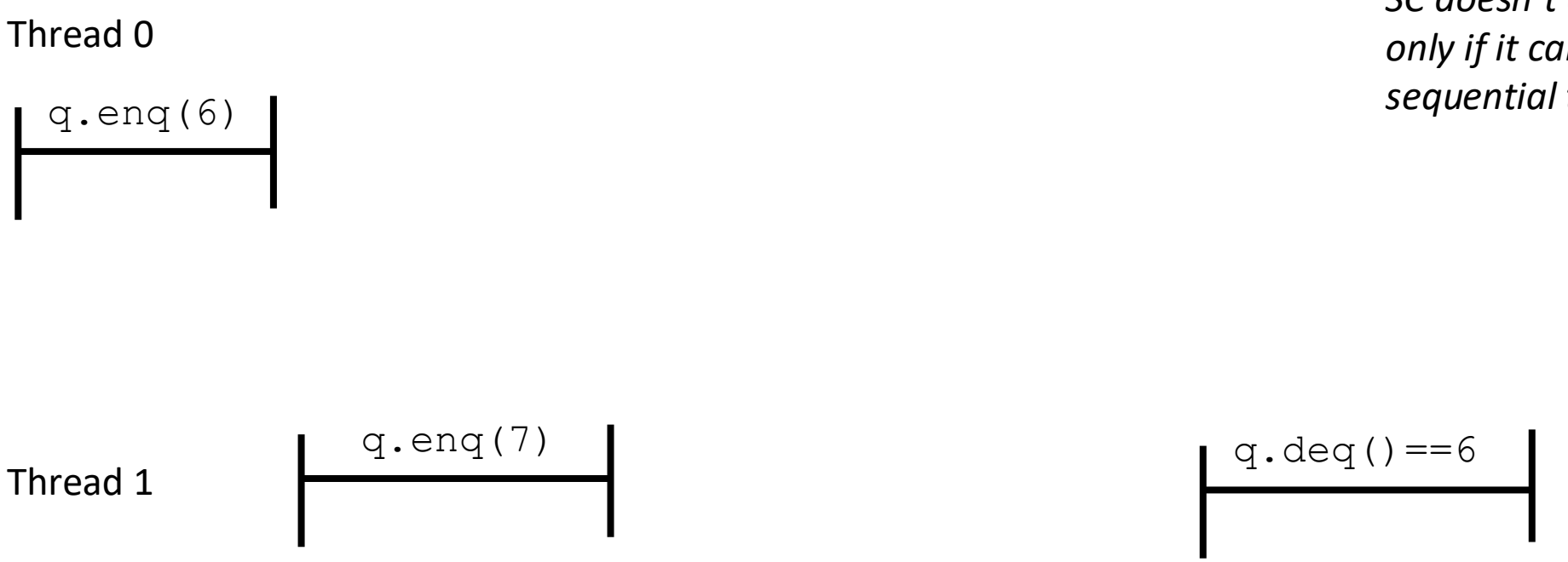


Sequential consistency and real time

- Add in real time:

This execution is allowed in sequential consistency!

SC doesn't care about real time, only if it can construct its virtual sequential timeline



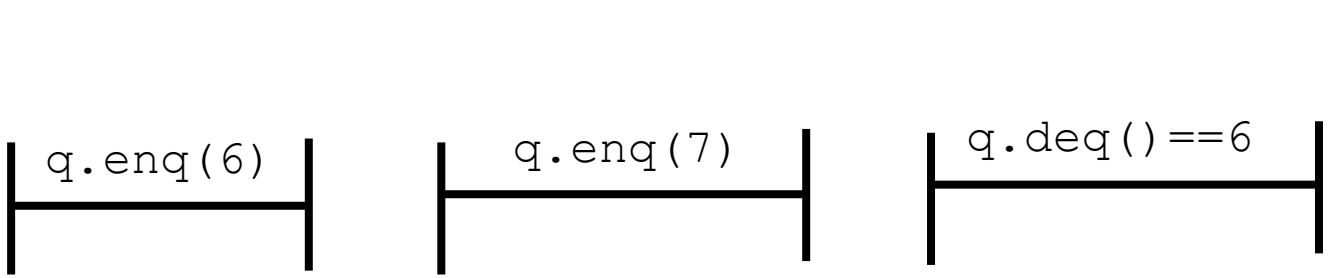
Sequential consistency and real time

- Add in real time:

This execution is allowed in sequential consistency!

SC doesn't care about real time, only if it can construct its virtual sequential timeline

Thread 0



Thread 1

